

RunPod Serverless Endpoint

FLUX.1-dev text-to-image, deployed and measured

Submitted by	Jimmie Young
Date	22 September 2026
Repository	https://github.com/jimmieyoung3/runpod-flux-serverless
Endpoint ID	ux61jghq0twkgq
Image	docker.io/jimmieyoung3/flux-runpod:v1 (private, 29.87 GB)
Model	black-forest-labs/FLUX.1-dev, bfloat16

Summary

A RunPod serverless endpoint that accepts a text prompt over HTTP and returns a generated image. The FLUX.1-dev weights are baked into the Docker image, so a worker loads the model from local disk in **7.2 seconds** rather than pulling 33.7 GB from the Hugging Face Hub. Warm generation of a 1024×1024 image at 28 steps takes **14.1 s on an A100** or **30.3 s on an A40**, at a cost of roughly **\$0.0103 per image** on either. Total cost of building, deploying and benchmarking the whole exercise was **\$0.85**.

The sections below cover the architecture, the handler and image design, two platform constraints that materially shaped the build, the measured results, and what I would change for production.



Endpoint output. Prompt: “a cinematic photograph of a glass terrarium on a windowsill at golden hour, shallow depth of field, 85mm”. Seed 42, 1024×1024, 28 steps.

1. Architecture

The endpoint is a single-purpose GPU worker. RunPod owns the queue, autoscaling and HTTP surface; the worker owns model loading and inference.

```
client -- POST /runsync {"input":{"prompt": ...}} --> RunPod queue
                                     |
                                     v
                                GPU worker (0 to 2, scale to zero)
                                |- import: FluxPipeline.from_pretrained(/models/flux)
                                |- warmup: one 512px / 1-step pass
                                \- per request: validate > generate > encode base64
```

Three decisions define the design.

Decision	Rationale	Measured effect
Weights baked into the image	Removes the Hub from the critical path; a cold worker reads from local disk.	Model ready in 7.2 s vs ~10 min from the Hub
Pipeline loaded at module import	RunPod attributes import time to worker start-up, not to billed request time.	Warm requests bill only inference
One job per worker (concurrency_modifier = 1)	Generation saturates the GPU; two concurrent 1024px jobs slow each other.	Execution stayed flat at 30.4 s under 2× load

A one-step warm-up pass runs at start-up so the first paying request does not absorb CUDA kernel autotuning and lazy initialisation.

2. The handler

src/handler.py is the RunPod entry point; src/predict.py owns the pipeline and src/schema.py validates input. Validation is deliberately free of torch and diffusers so it unit-tests on any machine. 25 tests run without a GPU.

```
def handler(job):
    params = validate(job.get("input"), DEFAULTS)
    result = predictor.generate(params)
    return {"images": [...], "seed": result.seed, "metrics": {...}}

runpod.serverless.start({"handler": handler,
                        "concurrency_modifier": adjust_concurrency})
```

Notable details

Behaviour	Why
Dimensions snap to a multiple of 16	FLUX packs latents 2×2 on top of the VAE's 8× downscale. Other values silently produce a differently sized image, so the handler rounds and reports rather than failing.
No negative_prompt	FLUX.1 is guidance-distilled and FluxPipeline has no negative conditioning. Silently accepting the field would be worse than rejecting it.
Errors return HTTP 200 with an error field	RunPod reserves non-2xx for platform failures; application errors belong in the body. Validation errors are separated from inference errors so a client mistake does not burn a retry.
Booleans rejected where numbers are expected	bool subclasses int in Python, so steps=True would otherwise silently mean 1 step.

Optional S3 delivery

Base64 keeps the endpoint dependency-free, but exceeds RunPod's ~20 MB response ceiling at about four 1024px PNGs. Setting `BUCKET_ENDPOINT_URL` switches to presigned URLs.

3. Two platform constraints

The image is 29.87 GB, which makes *where you build it* an engineering problem. Both constraints below were discovered during the build and are documented in the repository.

Buildah and Podman cannot run inside a RunPod pod

RunPod pods have no `CAP_SYS_ADMIN`, and the hosts set `apparmor_restrict_unprivileged_userns=1`, so `unshare(CLONE_NEWUSER)` is denied. Both tools re-exec into a user namespace at start-up, before they honour `--isolation chroot`, so even `buildah containers` fails. Kaniko uses no namespaces and is the workable builder. RunPod's own in-pod build tutorial reaches for Bazel and crane for the same reason.

Kaniko destroys the pod it runs in

Kaniko extracts the base image over `/`, deleting the userland that `sshd` depends on. SSH dies mid-build and does not recover; preserving the `sshd` binary via `--ignore-path` is not enough, because its PAM and `libc` dependencies go too. The consequence is architectural: Kaniko must **push the image itself**, because building to a local tarball strands the result on a pod you can no longer reach. The build runs detached under `setsid` and is monitored from outside against the registry.

Keeping the token out of the image

Kaniko does not implement `RUN --mount=type=secret`. Passing the Hugging Face token as a build argument would persist it in the image history, so instead the weights are downloaded into the build context beforehand and simply copied in. The token never enters the image, its history or its layers. That was verified by auditing the published image config, which contains zero token-shaped strings. The cost is peak build disk, since the weights exist twice during the build.

RunPod's GitHub integration would otherwise be the natural choice. It is ruled out on two counts: it caps `docker build` at 30 minutes, and it exposes no build-time secret, which a gated repository such as FLUX.1-dev requires.

Build pipeline as deployed

Stage	Where	Time
Fetch 33.7 GB of weights (79 files)	RunPod CPU pod, 32 vCPU	~3 min
Kaniko build + push of 29.87 GB	same pod, detached	~26 min
Total build cost	two pods, incl. one failed attempt	\$0.66

Root-level single-file checkpoints in the FLUX.1-dev repository duplicate the sharded `diffusers` folders that `FluxPipeline` actually loads. Excluding them cut the download from 57.8 GB to 33.7 GB.

4. Results

Measured against the live endpoint on 22 September 2026. `executionTime` is what RunPod bills; `delayTime` is queue plus worker start-up.

Latency at 1024×1024, 28 steps, guidance 3.5

Scenario	delayTime	executionTime	Wall
First-ever cold start, health call only	609 s	0.04 s	650 s
Cold start, image cached on host (A100)	15.5 s	16.0 s	35.2 s
Warm, A40 (mean of 5)	0.69 s	30.32 s (σ 0.06)	33.1 s
Warm, A100 (mean of 4)	0.80 s	14.07 s (σ 0.04)	16.0 s
2 concurrent, A40	16.42 s	30.40 s	49.9 s

The first row is a health call, which reports model and GPU information without generating, so its `executionTime` covers only the round trip. It is listed because its `delayTime` captures the one-time cost of scheduling a worker and pulling a 30 GB image onto a host that had never seen it.

A standard deviation of 0.06 s across warm runs confirms the pipeline stays resident and nothing is re-loaded per request. Model load from the baked weights is 7.2 to 8.0 s.

Cost

GPU	\$/hr	s/image	\$/image	Images per \$1
A40 48 GB	\$1.22	30.32	\$0.0103	97
A100-SXM4 80 GB	\$2.72	14.07	\$0.0106	93

The most useful finding: **cost per image is nearly GPU-independent**. A 2.2× faster card costs 2.2× more per second, so the two differ by 3%. Choose the GPU tier for the latency you need, not to save money. What actually moves cost is step count, resolution, and idle workers.

What a 30 GB image costs you

The first request sat in `throttled` for roughly 10 minutes before a worker was placed, and widening from 7 to 11 GPU tiers did not help, which points at the image rather than GPU scarcity. A host must cache ~30 GB before it can start a worker, a smaller pool than “any host with a free GPU”. So the baked-weights decision has a measurable cost as well as a measurable benefit:

	Baked weights (deployed)	Network volume (fallback)
Model available in	7.2 s	~10 min first boot, then volume read
First scheduling	~10 min throttle	fast, ~9 GB image
Runtime Hub dependency	none	first boot only

For a steady endpoint the baked image is the better trade. For one that scales from zero often, or across many regions, the volume variant schedules faster. It is implemented in the repository as `Dockerfile.volume`.

5. Sample generations

Unmodified endpoint output. All 28 steps, guidance 3.5.



Seed 7. "a hand-lettered enamel shop sign reading FLUX ON RUNPOD...". Text renders correctly.



Seed 1234. "an isometric cutaway of a tiny mechanical workshop... tilt-shift".



Seed 2026. "a lone lighthouse on a basalt cliff during a winter storm". Generated on the image-cached cold start, 15.5 s delay.



Seed 99, 832×1216, non-square output exercising the dimension snapping in schema.py.

6. Using the endpoint

```
curl -s -X POST https://api.runpod.ai/v2/ux61jghq0twkgq/runsync \
  -H "Authorization: Bearer $RUNPOD_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"input": {"prompt": "a red fox in a snowy forest", "seed": 7}}' > resp.json

python -c "import json, base64; d = json.load(open('resp.json'))['output']; open('out.png',
'wb').write(base64.b64decode(d['images'][0]))"
```

Or with the repository's client, which saves the decoded image and prints the metrics:

```
python client/call_endpoint.py "a red fox in a snowy forest" --seed 7
python client/call_endpoint.py --health # model + GPU report
python client/benchmark.py --runs 5 --gpu-rate 0.000339
```

Request fields

Field	Default	Notes
prompt	required	≤ 2000 characters
width, height	1024	snapped to a multiple of 16, clamped 256-1536
num_inference_steps	28	1-50
guidance_scale	3.5	0-20
seed	random	the seed used is always returned
num_images	1	1-4
output_format	PNG	PNG, JPEG or WEBP
action	(none)	"health" returns model/GPU info without generating

7. What I would change for production

Change	Why
Ship the network-volume variant across regions	The 30 GB image limits which hosts can place a worker. A ~9 GB image plus a pre-populated volume schedules faster and still avoids a per-boot download.
Return presigned S3 URLs, not base64	Base64 is convenient for review but inflates payloads by a third and hits RunPod's ~20 MB response ceiling at about four 1024px PNGs. The code path already exists.
Pin the GPU tier once traffic is known	Latency varied from 14 to 30 s purely on which card was scheduled. Latency-sensitive traffic should pin the faster tier; batch traffic should not pay for it.
Add request-level observability	The handler returns per-request metrics, but nothing aggregates them. Prometheus or a log sink would make regressions visible.
Quantised variants for throughput	fp8 or NF4 transformer weights roughly halve both load time and VRAM, at some quality cost. worth measuring against the bf16 baseline recorded here.

8. Knowledge base articles

Each problem hit during this build was written up as a customer-facing article in docs/kb/, in the shape a support article needs: what you see, what causes it, the commands that confirm it, and what to do about it. Where a cause is inferred rather than proven, the article says so.

Article	Issue it resolves
KB-001	Container build on a Pod fails with <code>unshare(CLONE_NEWUSER): Operation not permitted</code> . Includes the four commands that confirm it and the Kaniko workaround, with its side effects.
KB-002	Serverless workers stay throttled and jobs sit in the queue. How to tell GPU capacity apart from an image too large for hosts to cache.
KB-003	The first request to a new endpoint times out while later ones succeed. Why <code>/run</code> and polling beats <code>/runsync</code> when cold, and how to shorten cold starts.
KB-004	Keeping Hugging Face and registry credentials out of an image, including the case where BuildKit secrets are unavailable, and how to verify the published image rather than assume.

Full source, tests and deployment runbook: <https://github.com/jimmieyoung3/runpod-flux-serverless>

FLUX.1-dev weights are covered by the FLUX.1-dev Non-Commercial Licence, which is why the built image is held in a private registry and not redistributed.